ORBITAL_X PROJECT

FRONTEND ARCHITECTURE, 3D WEBGL STUDIO, REACT 19 UI & INCIDENT COPILOT

FRONTEND STACK:

- Framework: React 19 (v19.2.8) with React Router DOM (v7.18.3)
- Build Tool: Vite 8 (v8.2.2) with Rolldown vendor chunking
- Styling: Tailwind CSS v4 (@tailwindcss/vite) + Vanilla Cyber-Aerospace design system
- 3D Visualization: Three.js (v0.186.0), @react-three/fiber (v9.7.0), @react-three/drei (v10.7.8)
- Geospatial Globe: react-globe.gl (v2.38.0)
- Data Analytics & Charts: Recharts (v3.10.1)
- Icons: Lucide-React (v1.43.0)
- Default Listening Port: 4173

TABLE OF CONTENTS

1. ENTRY POINT & CORE APPLICATION SHELL

1.1 index.html, main.jsx & App.jsx

1.2 Styling Architecture (index.css & App.css)

1.3 State Contexts (AuthContext, SatelliteContext, ThemeContext)

2. API SERVICES LAYER (src/services/)

2.1 apiClient.js (Axios Instance, Bearer Token Interceptor & Error Normalizer)

2.2 Specialized Service Modules (All 12 Modules)

3. LAYOUT, NAVIGATION & SECURITY GUARDS

3.1 DashboardLayout.jsx, DashboardSidebar.jsx & DashboardHeader.jsx

3.2 ProtectedRoute.jsx, ErrorBoundary.jsx & PageLoader.jsx

4. 3D ORBITAL VIEWER & SHADER ENGINE (src/pages/orbit & src/components/3d/)

4.1 Earth.jsx, Scene.jsx & earthShaders.js (Dynamic Night/Day GLSL Shaders)

4.2 OrbitalViewer.jsx (Keplerian Visualizer, Ground Stations & Communication Arcs)

4.3 SatelliteMesh.jsx, OrbitalBottomDock.jsx & SatelliteInspectorHud.jsx

4.4 utils/orbitalMath.js (Keplerian Orbit Calculations & Coordinates)

5. 3D CUBESAT DESIGN STUDIO & SIMULATION ENGINE (src/pages/studio/)

5.1 CubeSatStudio.jsx & CubeSatCanvas.jsx (Interactive CAD Assembly Mode)

5.2 cubesatComponents.js (Avionics, Power, Comms, ADCS, Payload Specifications)

5.3 Procedural Geometry Generators:

- **buildChassisFrame.js (1U, 2U, 3U Anodized Aluminum Structural Rails)**
- **buildComponentMesh.js (PC104 Avionics, Battery Blocks, Reaction Wheels)**
- **buildSimSolarWings.js & buildSimPropulsionAndThermal.js**

5.4 Budgeting Engine (Mass, Volume, Power W, Eclipse Runtime & Cost Calculator)

5.5 CubeSatSimulator.jsx & CubeSatSimulationCanvas.jsx (Real-Time Physics Flight Mode)

5.6 Studio & Simulator Drawer / HUD Components (Hardware, Diagnostics, Commands)

6. OPERATIONAL PAGES & MISSION CONTROL

6.1 Dashboard.jsx (Constellation Metrics, Embedded Globe & Health Distribution)

6.2 SatelliteFleet.jsx & SatelliteDetail.jsx (Subsystem Telemetry & Orbit Details)

6.3 MissionFleet.jsx & MissionDetail.jsx (Apollo Operations & Assigned Links)

6.4 Communications.jsx (Encrypted Telecommand Dispatch & Signal SNR Console)

6.5 GroundStations.jsx (Antenna Coverage, Frequency Sweeps & Pass Countdown)

6.6 TelemetryAnalytics.jsx, FleetAnalytics.jsx & Reports.jsx (Recharts Telemetry)

7. INCIDENT MANAGEMENT, FAULT INJECTOR & AI COPILOT

7.1 AlertsIncidents.jsx (Incident Ledger & Filtering)

7.2 AnomalyInjectorDrawer.jsx (15+ Aerospace Failure Mode Injection Engine)

7.3 AutonomousAiSolverModal.jsx & AiDiagnosisCard.jsx (Root-Cause Analysis & 1-Click Dispatch)

7.4 data/satelliteAnomalyCatalog.js (Aerospace Failure Modes & Metric Thresholds)

7.5 NotificationCenter.jsx (Fleet Incident Decks & Urgent Feeds)

8. ADMIN HUB & SETTINGS

8.1 AdminDashboard.jsx & UserManagement.jsx (RBAC Role Elevation)

8.2 Profile.jsx, Preferences.jsx & Diagnostics.jsx (MongoDB & WebSocket Latency)

ENTRY POINT & CORE APPLICATION SHELL

1.1 index.html, main.jsx & App.jsx

- index.html:

Sets the aerospace cyber-dark background `#0A0F1F`, configures viewport meta tags, preconnects to

fonts, and mounts the React root `<div id="root"></div>`.

- src/main.jsx:

Initializes React 19 root using `createRoot(document.getElementById('root'))` and wraps the app

in `<React.StrictMode>`.

- src/App.jsx:

Configures route hierarchy with React Router v7. Wraps the app in:

- `<ThemeProvider>`

- `<AuthProvider>`

- `<SatelliteProvider>`

Implements route-level lazy loading (`React.lazy()`) for all primary views:

`/dashboard`, `/satellites`, `/satellites/:id`, `/orbit`, `/studio`, `/simulator`, `/missions`,

`/communications`, `/ground-stations`, `/telemetry`, `/analytics`, `/reports`, `/alerts`,

`/notifications`, `/admin`, `/admin/users`, `/admin/logs`, `/settings`.

1.2 Styling Architecture (index.css & App.css)

- Defines aerospace color tokens:

- Deep Space Canvas: `#050810`, `#0A0F1F`, `#070C18`

- Luminous Telemetry Cyan: `#00F0FF`

- Status Neon Green: `#00FF66`

- Caution Amber: `#FFB000`

- Critical Alarm Red: `#FF2A2A`

- Avionics Violet: `#C084FC`

- Glassmorphism Utilities:

Custom CSS classes for semi-transparent backdrops (`backdrop-blur-md`, subtle border glows,

radar sweep animations, scanline overlays).

1.3 State Contexts (src/context/)

- `AuthContext.jsx`:

Stores the authenticated user profile, token, and role. Manages `login(credentials)`,

`logout()`, and token persistence in `localStorage`.

- `SatelliteContext.jsx`:

Maintains active satellite selection across pages. When an operator selects a CubeSat in the fleet,

the selection propagates to the 3D Orbit Viewer, Studio, and Alert Drawer.

- `ThemeContext.jsx`:

Toggles between deep space dark mode and high-contrast tactical night vision.

API SERVICES LAYER (src/services/)

2.1 apiClient.js (Central Axios Engine)

- Creates the Axios instance with `baseURL: import.meta.env.VITE_API_URL || 'http://localhost:4000/api'`.

- Request Interceptor: Automatically inspects `localStorage` for `orbitalx_token` and attaches

`Authorization: Bearer <token>` to all outgoing requests.

- Response Interceptor: Intercepts HTTP 401 errors (expired tokens) and redirects the user to the login route.

2.2 Specialized Service Modules

- `authService.js`: `login()`, `register()`, `getMe()`.

- `satelliteService.js`: `getAll()`, `getById(id)`, `create(data)`, `update(id, data)`, `delete(id)`.

- `telemetryService.js`: `getLatest()`, `getHistorical(satId, range)`, `getFleetStats()`.

- `alertService.js`: `getAll(params)`, `getStats()`, `create(data)`, `acknowledge(id)`, `resolve(id)`.

- `aiService.js`: `diagnose(telemetry)` -> queries `/api/ai/diagnose` to trigger the Python Naive Bayes engine.

- `missionService.js`: `getAll()`, `getById(id)`, `create(data)`, `update(id, data)`.

- `groundStationService.js`: `getAll()`, `getById(id)`, `runDiagnostics(id)`.

- `communicationService.js`: `getTerminalLogs()`, `sendCommand(commandData)`, `exportCsv()`.

- `cubesatSimService.js`: `dispatchSimCommand()`, `fetchSimState()`.

- `notificationService.js`: `getAll()`, `markAsRead(id)`.

- `adminService.js`: `getUsers()`, `updateRole(id, role)`, `getAuditLogs()`, `setSimulationSpeed(speed)`.

- `analyticsService.js`: `getSummary()`, `getTrendMetrics()`.

LAYOUT, NAVIGATION & SECURITY GUARDS

3.1 DashboardLayout.jsx, DashboardSidebar.jsx & DashboardHeader.jsx

- `DashboardLayout.jsx`:

Two-column aerospace command shell. Houses the sticky sidebar on the left and scrollable page content

on the right with the header bar anchored at top.

- `DashboardSidebar.jsx`:

Collapsible navigation bar with cybernetic iconography (Lucide-React) for all modules:

Dashboard, Fleet, 3D Orbit, CAD Studio, Simulator, Missions, Comms, Ground Stations, Telemetry,

Alerts (with live red badge counter), Admin, and Settings.

- `DashboardHeader.jsx`:

Displays current Zulu time (UTC), global constellation status indicator (GREEN: 100% OPERATIONAL),

active satellite target dropdown, user profile pill, and emergency override button.

3.2 ProtectedRoute.jsx & ErrorBoundary.jsx

- `ProtectedRoute.jsx`:

Checks `isAuthenticated`. If false, redirects to `/login`. Optionally checks `requiredRole`

(e.g., restricting `/admin` to `ADMIN` users).

- `ErrorBoundary.jsx`:

React class component with `componentDidCatch`. Prevents WebGL canvas crashes from tearing down

the entire application, rendering a cyber-styled "RELOAD SUBSYSTEM" fallback instead.

3D ORBITAL VIEWER & SHADER ENGINE (src/components/3d/)

4.1 Earth.jsx, Scene.jsx & earthShaders.js

- `Earth.jsx`:

Constructs a procedural WebGL globe using Three.js sphere geometry.

Applies multi-texture shaders: day diffuse map, night city lights emissive map, bump map,

and atmospheric glow haze.

- `earthShaders.js`:

Custom GLSL vertex and fragment shaders. Calculates sunlight direction vector and blends day

diffuse texture with night city lights along the terminator line using smoothstep functions:

```
float sunIntensity = dot(vNormal, uSunDirection);

vec4 dayColor = texture2D(uDayTexture, vUv);

vec4 nightColor = texture2D(uNightTexture, vUv);

gl_FragColor = mix(nightColor, dayColor, smoothstep(-0.1, 0.1, sunIntensity));
```

- `Scene.jsx`:

Renders the Three.js Canvas with OrbitControls, starfield background particles, ambient lighting,

and directional solar lighting.

4.2 OrbitalViewer.jsx

- Plots orbits for all active constellation satellites.

- Computes Keplerian elliptical curves from semi-major axis, eccentricity, inclination, and RAAN.

- Renders global ground tracking stations as pulsed cyan antennas on the globe.

- Draws parabolic Bezier communication arcs between satellites and ground stations when within line of sight.

3D CUBESAT DESIGN STUDIO & SIMULATION ENGINE (src/pages/studio/)

5.1 CubeSatStudio.jsx & CubeSatCanvas.jsx (CAD Assembly Canvas)

- Supports standard **1U, 2U, and 3U** CubeSat chassis configurations.

- Assembly Modes:

  - `ASSEMBLE`: Exploded view animation where modules slide smoothly outward along their mounting rails.

  - `VALIDATE`: Sweeps a luminous cyan laser line vertically through the CubeSat chassis, inspecting

    connector alignments and mass/power budget constraints.

  - `NORMAL`: Compact operational flight configuration.

5.2 cubesatComponents.js & Component Library

Defines physical specifications for components:

- Avionics: PC104 On-Board Computers (Mass: 90g, Power: 0.8W, Vol: 0.2U)

- Power: Triple-junction GaAs Solar Panels (Power Gen: +7.2W) & Li-Ion Battery Pack Block (120g, Vol: 0.3U)

- Comms: Deployable Spring Whip Antenna & S-Band Transmitters (1.2W)

- ADCS: 3-Axis Reaction Wheels (Brass Flywheels) & Magnetorquer Rods

- Payload: Nadir Optical Imager Camera & Multispectral Radiation Spectrometer

5.3 Procedural Mesh Generators (src/pages/studio/utils/)

- `buildChassisFrame.js`:

  Procedurally constructs standard 10x10x10cm (1U), 10x10x20cm (2U), or 10x10x30cm (3U) structural

  extrusion rails using Three.js BoxGeometry and anodized black aluminum metallic materials.

- `buildComponentMesh.js`:

  Constructs detailed 3D meshes for components: green PCB substrates, gold-plated headers, copper coils,

  brass flywheel discs, and optical lenses.

- `buildSimSolarWings.js`:

  Generates deployable accordion solar array wings with dark blue photovoltaic cell textures.

- `buildSimPropulsionAndThermal.js`:

Generates cold-gas micro-thruster nozzles, heat straps, and radiator plates.

5.4 Real-Time Physics Budgeting Engine

Calculates physical flight margins dynamically as parts are added:

- Total Mass (g) vs. Launch Adapter Payload Limit (max 4000g for 3U)

- Total Volume (U) vs. Chassis Envelope

- Power Generation (+W) vs. Operational Load (-W)

- Eclipse Battery Runtime (minutes in Earth shadow before minimum voltage cutoff)

- Bill of Materials (BOM) total cost ($) with instant CSV export.

5.5 CubeSatSimulator.jsx & CubeSatSimulationCanvas.jsx

- Real-time physics simulator placing the assembled CubeSat in orbit.

- Simulates solar exposure based on attitude, reaction wheel desaturation maneuvers, and telemetry streams.

- SimCommandConsole allows operators to dispatch telecommands directly into the simulated flight computer.

OPERATIONAL PAGES & MISSION CONTROL

6.1 Dashboard.jsx

- Mission overview displaying:
  - Constellation Health Metric (e.g. 98.4%)
  - Active Satellites (LEO: 6, MEO: 1, GEO: 1)
  - Orbital Coverage Gauge (Recharts Circular Gauge)
  - Active Anomaly & Incident Counter
  - Embedded 3D Earth Snapshot
  - Real-Time Telemetry Stream Feed

6.4 Communications.jsx (Encrypted Telecommand Console)

- Encrypted telecommand uplink/downlink terminal.
- Three-column layout:
  - Column 1: Active Satellite Targets & Line-of-Sight indicators.
  - Column 2: Live Encrypted Terminal with priority dispatch (`NORMAL`, `HIGH`, `EMERGENCY`).
  - Column 3: Real-time RF signal strength gauge (-62 dBm), link latency (124ms), and AES-256 verification.

6.6 TelemetryAnalytics.jsx & FleetAnalytics.jsx

- Recharts historical area and line charts:
  - Solar array power vs. battery state of charge
  - Core CPU and battery pack temperature trends
  - Signal-to-noise ratio (SNR) and RF reflected power
- Provides 24-hour summary reports and CSV data export.

INCIDENT MANAGEMENT, FAULT INJECTOR & AI COPILOT

7.1 AlertsIncidents.jsx

- Central incident triage center. Displays active red alarms, amber warnings, and resolved history.
- Features embedded 30-minute sparkline charts demonstrating nominal telemetry before failure spikes.

7.2 AnomalyInjectorDrawer.jsx

- Slide-out drawer allowing flight directors to inject simulated aerospace emergencies:
  - Filter by subsystem tabs: `ALL`, `EPS`, `ADCS`, `COMMS`, `TCS`, `OBC`, `ENV`, `PAYLOAD`, `CUSTOM`.
  - Target CubeSat selector dropdown.
  - Clicking `[INJECT]` immediately dispatches the anomaly to the backend and triggers telemetry spikes.

7.3 AutonomousAiSolverModal.jsx & AiDiagnosisCard.jsx

- Incident copilot that queries the Python AI microservice.

- Displays:

  - Failure Mode & Confidence Score (e.g. 98.2%)

  - Classification Domain (`INNER_ERROR` vs `OUTER_COLLISION_DAMAGE`)

  - Root-Cause Physical Explanation

  - SOP Checklist with actionable checkboxes

  - Large Cyan `[EXECUTE 1-CLICK MITIGATION TELECOMMAND]` button (e.g. `CMD_EPS_SAFE_MODE`).

    Clicking this button transmits the telecommand, marks the alert resolved, and resets subsystem telemetry.

7.4 data/satelliteAnomalyCatalog.js

- Defines the 16+ preset aerospace anomalies, metrics, thresholds, and pre-programmed telecommands.

ADMIN HUB & SETTINGS

8.1 AdminDashboard.jsx & UserManagement.jsx

- User privilege administration table: elevate users to `ADMIN`, `OPERATOR`, `ANALYST`, or `VIEWER`.

- Simulation rate slider: modify telemetry interval from 1s to 60s.

- Immutable security audit log table tracking all telecommands and logins.

8.2 Profile.jsx, Preferences.jsx & Diagnostics.jsx

- Displays operator access keys, UI rotation speed controls, night-vision defaults.

- Diagnostics panel displays live ping for REST API, MongoDB latency, and WebSocket socket ping.